

מדריך הפרויקט — investment-research-rag

תיעוד מלא של מערכת חיפוש המידע על השקעות. מסמך זה נועד כגיבוי וכמדריך שחזור. אם המחשב יקרוס, או אם תחזור לפרויקט אחרי חודשים — כאן יש את הכל.

תוכן עניינים

- סקירה כללית — מה זה ולמה
- מצב נוכחי — מה כבר עובד
- איך זה עובד — RAG בעברית פשוטה
- המקומות שבהם הכל יושב
- שימוש יומיומי — איך לשאול ולהוסיף ספרים
- שחזור מאפס — אם המחשב יקרוס
- עלויות — מה הוצאתי, מה ההמשך
- פירוט הקבצים בפרויקט
- שלב ב' — GraphRAG (העתיד)
- פתרון תקלות נפוצות
- פרטי חשבונות וגישה

1. סקירה כללית

מה זה הפרויקט?

מערכת אישית לחיפוש מידע בספריית ספרים פיננסיים. במקום לחפש ידנית בעשרות ספרים, אתה שואל שאלה בעברית או באנגלית, והמערכת:

- מבינה את השאלה
- שולפת קטעים רלוונטיים מכל הספרים שהעלית
- שולחת אותם ל-Claude (AI) שיענה
- מצטטת בדיוק מאיזה ספר ועמוד הגיעה כל אמירה

למה זה שווה?

- **אין בלוף** — Claude עונה רק ממה שכתוב בספרים שלך, לא ממקורות חיצוניים
- **ציטוטים** — כל תשובה מגיעה עם מקור מדויק (ספר + עמוד)
- **דו-לשוני** — תומך בעברית ובאנגלית
- **פרטי** — הספרים שלך, התשובות שלך — לא יוצא לאף אחד חוץ מ-OpenAI/Anthropic ל-API calls
- **זול** — כ-2 סנט עלות חד-פעמית להכנסת 1,000 chunks; שאלה עולה פחות מסנט

שני שלבים

- **שלב א' (הושלם)** ☒: RAG בסיסי — חיפוש סמנטי על ספרים והחזרת ציטוטים
- **שלב ב' (תוכנן, עתידי)**: GraphRAG — הוצאת ישויות (חברות, אנשים, מושגים) והקשרים ביניהן לגרף ידע

2. מצב נוכחי

נכון לתאריך 23.05.2026:

מדד	ערך
ספרים שנקלטו	6 ייחודיים (7 קבצים, אחד היה כפול)
Chunks במסד	1,026
שפת הספרים	אנגלית (כולם על Warren Buffett)
מצב Phase 1	☒ עובד מקצה לקצה
Repo ב-GitHub	https://github.com/lihi94/investment-research-rag
מסד נתונים	Supabase project <code>knngijdnvyswmzffrmlr</code> (eu-central-1)

הספרים הקיימים במערכת:

1. The Essays of Warren Buffett: Lessons for Corporate America — Warren Buffett
2. Pick Stocks Like Warren Buffett — Warren Boroson
3. The Warren Buffett Portfolio — Robert G. Hagstrom
4. The Warren Buffett Way — Robert G. Hagstrom
5. Trade Like Warren Buffett — James Altucher
6. Warren Buffett on Business Principles from the Sage of Omaha — Richard J. Connors

3. איך זה עובד

הבעיה שפתרנו

LLM-ים כמו Claude לא יודעים על הספרים האישיים שלך. אם תשאל אותם "מה Buffett אמר על X", הם יענו ממידע ציבורי כללי, ולא בהכרח מהספרים שיש לך.

הפתרון: RAG (Retrieval-Augmented Generation)

במקום לבקש מ-Claude לזכור את כל הספרים (לא אפשרי), אנחנו עושים שלושה דברים:

שלב 1: הכנה חד-פעמית (`02_ingest.py`)

PDF → חתיכות → טקסט (chunks) → embeddings → Supabase

- כל ספר מחולק ל-chunks של ~650 מילים (עם 80 מילים חפיפה, כדי שלא נחתוך אמצע משפט)
- כל chunk מומר ל-**embedding** — וקטור מספרי באורך 1,536 שמייצג את "המשמעות" של הטקסט
- ה-embeddings נשמרים ב-Supabase עם **pgvector** (תוסף שמאפשר חיפוש דמיון בין וקטורים)

שלב 2: שאילתא (`03_query.py`)

הכי דומים 8 → chunks-של השאלה → חיפוש דמיון בכל ה-embedding → שאלה
↓
"ענה רק מהקטעים האלה" context-עם ה Claude-שולח ל
↓
'תשובה עם ציטוטים [#1], [#3], וכו'

למה זה עובד?

ה-embedding של "value investing" יהיה דומה ל-embedding של "השקעות ערך" וגם דומה לקטעים שמדברים על "buying undervalued stocks". אז גם אם השאלה שלך לא מכילה את אותן מילים בדיוק, המערכת תמצא קטעים רלוונטיים.

4. המקומות שבהם הכל יושב

🖥️ במחשב שלך:

נתיב הפרויקט:

```
C:\Users\eliha\OneDrive\Desktop\cluade code\investment-research-rag\
```

מבנה התיקיות:

investment-research-rag/

├── .env

├── .env.example

├── .gitignore

├── README.md

├── PROJECT_GUIDE.md

├── requirements.txt

├── data/pdfs/

├── logs/

├── .venv/

├── db/

├── src/

├── scripts/

🔒 API (לא ב git)

(git-כן ב) .env תבנית ריקה של

git קבצים שלא עולים ל

תיעוד בסיסי לפרויקט

המסמך הזה ←

Python רשימת חבילות

(git-לא ב) הספרים

(git-לא ב) לוגים של הקליטה

(git-לא ב) Python סביבה וירטואלית של

סכמת מסד הנתונים

לשימוש חוזר Python מודולים

.env-טעינת מפתחות מ

Supabase של wrapper

PDF-חילוץ טקסט מ

חיתוך לחתיכות

OpenAI embeddings של wrapper

סקריפטים להרצה ישירה

סקר ראשוני (אומדן עלויות, לא נוגע במסד)

לתוך המסד PDFs קליטת

לשאלות אינטראקטיות CLI

☁️ ב-Supabase (ענן):

Project: knggijdnvyswmzffrmlr **URL:** <https://knggijdnvyswmzffrmlr.supabase.co> **Dashboard:** <https://supabase.com/dashboard/project/knggijdnvyswmzffrmlr>

טבלאות: - **sources** — מטא-דאטה של כל ספר (שם, מחבר, ingested_at, hash) - 1,026 **chunks**
חתיכות עם ה-embeddings (1536-ממדים, אינדקס HNSW) - **entities** — ריקה, מוכנה לשלב ב' -
relationships — ריקה, מוכנה לשלב ב'

פונקציה `match_chunks(query_embedding, threshold, count)` - **SQL:** - מקבלת embedding ומחזירה את ה-N הכי דומים

 **ב-GitHub:**

Repo: <https://github.com/lihi94/investment-research-rag> **Branch:** `main` **Visibility:** Public
(שנות ל-Private ב-Danger Zone → Settings)

מה יש שם: כל הקוד + תיעוד. **אין שם:** מפתחות venv, logs, PDFs, API.

5. שימוש יומיומי

לפתוח את הפרויקט מחדש (כל פעם):

```
cd "C:\Users\eliha\OneDrive\Desktop\cluade code\investment-research-rag"
.venv\Scripts\activate
```

(אתה תראה `.venv`) בתחילת השורה. זה אומר שהסביבה הווירטואלית פעילה.)

לשאל שאלה:

```
python scripts/03_query.py
```

תקבל `?` prompt ופשוט תקליד שאלה. דוגמאות: - `What is Warren Buffett's view on dividends?`

- `Explain the margin of safety concept` - `?מה ההבדל בין השקעת ערך לצמיחה`

ליציאה: `q` או `quit` או `exit`.

להוסיף ספרים חדשים:

1. **שים את הקבצים** ב- `data/pdfs/` (אפשר ליצור תתי-תיקיות)

2. **הרץ סקר** כדי לראות את העלות הצפויה: `powershell python scripts/01_survey.py`

3. **קלוט:** `powershell python scripts/02_ingest.py` - הסקריפט אוטומטית מדלג על ספרים שכבר

קלטת (לפי SHA256 hash) - אפשר להפסיק (Ctrl+C) ולהמשיך — Resumable

הוספת ספרים סרוקים (PDF שזה תמונות, לא טקסט):

צריך **OCR**. תתקין: - **Tesseract** מ- <https://github.com/UB-Mannheim/tesseract/wiki> (סמן + Hebrew

- **Poppler** (English) מ- <https://github.com/oschwartz10612/poppler-windows/releases>

ואז הרץ:

```
python scripts/02_ingest.py --ocr
```

לשמור שינויים ב-GitHub:

אחרי שינוי קוד או הוספת תיעוד:

```
git add -A
git commit -m "תיאור השינוי"
git push
```

לבדוק מצב המסד:

ב-GitHub → Table Editor → sources או chunks . או דרך SQL:

```
SELECT * FROM source_stats;
```

6. שחזור מאפס

מצב: המחשב התרסק / קנית מחשב חדש / רוצה להתחיל מחדש.

מה אבוד ומה לא:

פריט	אבוד?	היכן?
הקוד	לא ❌	GitHub
מסד הנתונים + 1,026 chunks	לא ❌	Supabase (בענן)
ה-PDF-ים	אולי ⚠️	OneDrive/Google Drive — תלוי איפה שמרת
.env (מפתחות API)	אולי ⚠️	תלוי איפה שמרת
.venv	אבד ✅	אבל זה רק חבילות, אפשר להתקין מחדש

שלבי שחזור:

שלב 1: התקנת תוכנות יסוד (אם המחשב חדש)

1. **Python 3.12** (לא 3.13 או 3.14 — חבילות מסוימות לא תומכות): - הורד מ-

<https://www.python.org/downloads/release/python-3128> - בהתקנה: סמן "Add Python to PATH"

2. **Git:** <https://git-scm.com/download/win>

3. **VS Code** (אופציונלי): <https://code.visualstudio.com/>

שלב 2: שכפול הפרויקט מ-GitHub

```
cd "C:\Users\שם המשתמש שלך\Desktop"
git clone https://github.com/lihi94/investment-research-rag.git
cd investment-research-rag
```

שלב 3: יצירת סביבה וירטואלית והתקנת חבילות

```
py -3.12 -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

שלב 4: מילוי `.env`

```
copy .env.example .env
notepad .env
```

תצטרך למלא: - `SUPABASE_URL` — כבר ממולא (-) (<https://knggijdnvyswmzffrmlr.supabase.co>)
— `SUPABASE_SERVICE_ROLE_KEY` — מ-Dashboard → Project Settings → API Keys → **Legacy** tab
— `OPENAI_API_KEY` - `service_role` — אם לא שמרת, צריך ליצור חדש ב-<https://platform.openai.com/api-keys>
— `ANTHROPIC_API_KEY` - `keys` — אם לא שמרת, צור חדש ב-<https://console.anthropic.com/settings/keys>

שלב 5: שים את ה-PDFs

<הספריים> `data/pdfs/`. אם הם בגוגל-דרייב, פשוט הורד.

שלב 6: בדיקה

```
python scripts/03_query.py
```

אם השאלה הראשונה עובדת ומחזירה ציטוטים — הכל עובד. המסד עדיין מלא מהפעם הקודמת.

7. עלויות

עלויות חד-פעמיות שכבר נצרכו:

שירות	מה שילמנו	על מה
OpenAI Embeddings	\$0.02~	הטמעה של 1,026 chunks (~1M tokens)
Anthropic	משתמש מהקרדיט שטענת	שאלות (כל שאלה ~\$0.005-0.02)
Supabase	\$0 (Free tier)	המסד עדיין קטן מאוד

עלויות צפויות בהמשך:

אם תוסיף את 200 הספרים שתכננת: - הטמעה חד-פעמית: ~\$2-5 (תלוי באורך הספרים) - אחסון Supabase: עדיין בתוך ה-Free tier (עד 500MB; אנחנו עכשיו רחוקים מזה)

שאלות שוטפות: - Claude (sonnet-4-6): ~\$0.005-0.02 לשאלה - 100 שאלות בחודש = ~\$0.50-\$2

יעד תקציבי: \$5-10 לחודש סביר אם תשתמש באופן רגיל.

איך לעקוב אחרי הוצאות:

- OpenAI:** <https://platform.openai.com/usage>
- Anthropic:** <https://console.anthropic.com/settings/billing>
- Supabase:** Dashboard → Settings → Billing

טיפ לחיסכון:

ב- `scripts/03_query.py` השורה `TOP_K = 8` קובעת כמה chunks נשלפים. אם תוריד ל-5, השאלות יהיו זולות יותר (פחות טקסט נשלח ל-Claude), בעלות איכות תשובה אולי קצת פחות עמוקה.

8. פירוט הקבצים בפרויקט

קבצי תצורה

קובץ	מה הוא עושה
<code>.env</code>	מפתחות API + נתיבים. לעולם לא ב-git!
<code>.env.example</code>	תבנית של (placeholders) <code>.env</code> . דווקא כן ב-git.
<code>.gitignore</code>	מה לא להעלות ל-venv, logs, PDFs, <code>.env</code> (git)
<code>requirements.txt</code>	חבילות Python שצריך
<code>README.md</code>	תיעוד התקנה בסיסי (בעיקר ל-GitHub)
<code>PROJECT_GUIDE.md</code>	המסמך הזה

`db/schema.sql`

הסכמה המלאה של מסד הנתונים. אם תרצה לשחזר את המסד מאפס (למשל ב-Supabase project חדש): 1. צור project חדש ב-Supabase 2. SQL Editor → New query → הדבק את התוכן של `Run 3` → `schema.sql`. הסכמה כוללת: 4 טבלאות, 2 אינדקסי HNSW, פונקציית `source_stats`, view `match_chunks`.

`src/config.py`

טוען את `.env` ויוצר אובייקט `CONFIG` שכל הסקריפטים משתמשים בו. בודק שכל המפתחות קיימים בעת הטעינה — אם חסר משהו, יוצא עם שגיאה ברורה.

`src/db.py`

עטיפה ל-Supabase client. הפונקציות העיקריות: - `find_source_by_hash()` — בודק אם ספר כבר נקלט (לפי SHA256) - `mark_source_ingested()` / `insert_chunks()` / `insert_source()` — כתיבות - `match_chunks()` — קריאת ה-RPC לחיפוש דמיון

`src/pdf_utils.py`

חילוך טקסט מ-hash — `sha256_file()` - PDFs של קובץ (לזיהוי כפילויות) - `detect_pdf_kind()` — סורק 5 דפים ומחליט: Generator — `extract_pages()` - `'text'` / `'scanned'` / `'mixed'` שמחזיר עמוד-עמוד (עם דילוג על דפים סרוקים, אלא אם `OCR` — `_ocr_single_page()` - `use_ocr=True` עם Tesseract (lazy import — לא נטען אלא אם משתמשים)

`src/chunker.py`

חיתוך הטקסט לחתיכות. אלגוריתם: 1. שטיחת העמודים למשפטים (תוך שמירת מספר עמוד) 2. אריזת משפטים לחתיכות של ~650 מילים (Greedy) 3. חפיפה של 80 מילים בין חתיכות (כדי לא לאבד הקשר בגבול) **למה לא חותכים באמצע משפט:** המשפט הוא יחידת המשמעות. חיתוך באמצע משפט מקלקל את ה-embedding.

`src/embeddings.py`

עטיפה ל- `count_tokens()` - OpenAI Embeddings — לפני שליחה (לתמחור ולוודא שלא חורגים מהמגבלה של 8,192 tokens לקלט) - `embed_batch()` — שליחה בודדת עם retry אקספוננציאלי (3 ניסיונות, המתנה 2/4/8 שניות) - `embed_many()` — מקבל list בכל גודל ומחלק לבאצ'ים אוטומטית (עד 100 פריטים או 250k tokens לבאצ')

`scripts/01_survey.py`

מטרה: לפני קליטה, לראות כמה זה יעלה ואיפה יש בעיות.
מה הוא עושה: - סופר PDFs, גודל כולל - מסווג כל PDF: digital / scanned / mixed - ל-digital מבצע חילוך + chunking (בלי לשלוח ל-OpenAI!) כדי להעריך כמות tokens - מציע הערכת עלות
מה הוא לא עושה: לא נוגע במסד, לא שולח API calls בתשלום.

`scripts/02_ingest.py`

מטרה: קליטת ה-PDFs למסד.

הזרימה לכל קובץ: 1. SHA256 → בדיקת 2 dedup. אם נקלט כבר ב-100% → דלג 3. אם נתקע באמצע מהריצה הקודמת → נקה chunks ישנים, חזור 4. אחרת: צור record חדש ב-5 `sources`. חלץ עמודים → צ'אנקים → embedding בבאצ'ים של 64 → הכנס 6. סמן `ingested_at = now()`

Flags: - `--ocr` — הפעל OCR לדפים סרוקים (איטי, דורש התקנה של Tesseract + Poppler) - `--limit N` — קלוט רק N קבצים (לבדיקות)
Logs: נכתבים ל- `logs/ingest.log`.

`scripts/03_query.py`

מטרה: CLI אינטראקטיבי לשאלות.

הזרימה לכל שאלה: 1. embed את השאלה 2. 3 `match_chunks(top_k=8, threshold=0.3)`. בנה prompt עם הקטעים ממספרים `[#1]`, `[#2]` ... 4. שלח ל-Claude עם system prompt קפדני: "ענה רק מההקשר. צטט `[N#]`". 5. הזרם את התשובה לקונסול 6. הדפס את רשימת המקורות עם דמיון

הגדרות שניתן לכוון בראש הקובץ: - `TOP_K = 8` — כמה chunks לשלוף - `SIMILARITY_THRESHOLD = 0.3` — סף דמיון מינימלי (0.3 מתירני, 0.5 מחמיר)

9. שלב ב' GraphRAG

מה זה?

במקום לחפש רק קטעים דומים, גם נבנה **גרף ידע**: ישויות (חברות, אנשים, מושגים פיננסיים) והקשרים ביניהם.

דוגמה: - ישויות: `Warren Buffett`, `Berkshire Hathaway`, `Value Investing`, `Margin of Safety`
קשרים: `Buffett` —[CEO_OF]→ `Berkshire`, `Buffett` —[ADVOCATES]→ `Value Investing`

למה זה שווה?

שאלות מורכבות יותר נעשות אפשריות: - "מי החברות ש-Buffett השקיע בהן ומה היה הרציונל?" - "אילו עקרונות חוזרים על עצמם בכל הספרים?" - "תראה לי את כל הקשרים בין X ל-Y"

מה כבר מוכן?

הטבלאות `entities` ו-`relationships` כבר קיימות ב-Supabase. גם השדה `source_chunk_id` שמקשר כל ישות חזרה ל-chunk שממנו היא הוצאה.

מה צריך לבנות?

1. **סקריפט הוצאת ישויות:** עובר על כל chunk, שולח ל-Claude עם prompt שמבקש לזהות ישויות וקשרים, מחזיר JSON
2. **דה-דופליקציה:** "Warren Buffett" ו-"W. Buffett" ו-"באפט" צריכים להיות אותה ישות
3. **embedding לישויות** (השדה כבר במסד) — לחיפוש סמנטי של ישויות
4. **CLI שאילתות גרפי:** משלב חיפוש `chunks + traversal` של הגרף

הערכת עלות שלב ב':

הוצאת ישויות מ-1,000 chunks ב-\$3-5 ≈ Claude Haiku 4.5 חד-פעמי. אם נוסיף עוד ספרים — פרופורציונלי.

10. פתרון תקלות נפוצות

"Invalid API key" כשמריצים סקריפט

בדוק: האם `.env` קיים בתיקיית הפרויקט (ולא ב-תת-תיקייה)? **בדוק:** האם המפתח מועתק ללא רווחים / שורה ריקה בסוף? **בדוק:** האם זה המפתח הנכון? Supabase יש 2 סוגים — `service_role` (אנחנו), לא

"Permission denied" ב-Supabase

סיבה: RLS פעיל. ב- `db/schema.sql` כתבנו `DISABLE ROW LEVEL SECURITY`, אבל אם משחזרים את המסד וזה לא מופעל — תפעיל ידנית:

```
ALTER TABLE sources DISABLE ROW LEVEL SECURITY;
ALTER TABLE chunks DISABLE ROW LEVEL SECURITY;
ALTER TABLE entities DISABLE ROW LEVEL SECURITY;
ALTER TABLE relationships DISABLE ROW LEVEL SECURITY;
```

נכשל על תוכנת קומפילציה `pip install`

סיבה: Python 3.13/3.14 — לחלק מהחבילות אין wheels מוכנים, והם מנסים לקמפל מהמקור. **פתרון:** השתמש ב- `py install 3.12`, Python 3.12, ואז `.venv -m venv -3.12 py`.

"0 chunks extracted" למרות שה-PDF נראה תקין

סיבה אפשרית 1: OneDrive עוד לא הוריד את הקובץ (cloud-only stub). **פתרון:** Right-click על התיקיה → "Always keep on this device".

סיבה אפשרית 2: ה-PDF סרוק (תמונות) — צריך OCR. **בדיקה:** `python scripts/01_survey.py` — יסווג את כל הקבצים.

השאלות מחזירות "no relevant context found"

סיבה: סף הדמיון גבוה מדי. ב- `scripts/03_query.py` תוריד את `SIMILARITY_THRESHOLD` ל-0.2. **או:** הספרים פשוט לא מכסים את הנושא. הוסף עוד מקורות.

Push ל-GitHub נכשל ("Authentication failed")

ב-Git, Windows, משתמש ב-**Credential Manager**. אם בפעם הראשונה התחברת — אמור לעבוד אוטומטית. אם לא:

```
git config --global credential.helper manager
git push
# GitHub של login יפתח חלון
```

11. פרטי חשבונות וגישה

חשבונות שצריך:

שירות	למה	URL
lihi94 (GitHub)	אחסון קוד	https://github.com/lihi94
Supabase	מסד נתונים	https://supabase.com/dashboard
OpenAI Platform	API ל-embeddings	https://platform.openai.com
Anthropic Console	API ל-Claude	https://console.anthropic.com

היכן לשמור מפתחות API:

המפתחות נמצאים ב- `.env` במחשב, שלא עולה ל-GitHub (זה מאובטח).
אם תרצה גיבוי של המפתחות: לא ב-Google Keep / WhatsApp / Email — לא מאובטח - כן ב- Password Manager (2FA עם Bitwarden, 1Password, Apple Keychain, Chrome password manager)

אם איבדת מפתח:

- OpenAI: https://platform.openai.com/api-keys → Create new secret key
 - Anthropic: https://console.anthropic.com/settings/keys → Create key
 - Supabase: Dashboard → Project Settings → API → Legacy tab → service_role
- אחרי יצירת מפתח חדש, מחק את הישן (אם מישהו אחר עלול להחזיק אותו).

נספח: פקודות שימושיות לזיכרון מהיר

עבודה יומיומית

```
# פתיחה
cd "C:\Users\eliha\OneDrive\Desktop\cluade code\investment-research-rag"
.venv\Scripts\activate

# שאלה
python scripts/03_query.py

# data/pdfs/ ל-PDFs אחרי שהכנסת) הוספת ספרים חדשים
python scripts/02_ingest.py

# סקר עלות לפני קליטה
python scripts/01_survey.py
```

Git

git status	# מה השתנה
git add -A	# staging-להוסיף הכל ל
git commit -m "ליצור"	# commit תיאור השינוי"
git push	# GitHub-להעלות ל
git log --oneline	# commits היסטוריית
git pull	# (אם עבדת ממחשב אחר) GitHub-למשוך שינויים מ

Supabase דרך SQL Editor

```
-- יש לכל ספר chunks כמה --
SELECT * FROM source_stats;

-- כל הספרים
SELECT id, title, ingested_at FROM sources ORDER BY ingested_at DESC;

-- (chunks-ימחק את ה cascade) למחוק ספר --
DELETE FROM sources WHERE title = '<שם הספר>';
```